

SPŠE Ječná

Informační technologie

Ječná 30, 120 00, Praha 2

Dokumentace k projektu

Lunar Lander

Matyas Potocký

IT

2025

Obsah

1	Cíl práce	2
2	Popis práce	2
2.1	Herní mechaniky	2
2.2	Uživatelské rozhraní a průběh hry	3
2.3	Algoritmy	4
3	Systémové požadavky (System requirements)	4
4	Základní struktura	4
5	Testovací data	6
5.1	Jednotkové testy (Unit Tests)	6
5.2	Manuální testování	6
6	Analýza problému nefunkčního unit testu	7
6.1	Popis chyby z výstupu testu	7
6.2	Pravděpodobné příčiny problému	10
6.3	Možné kroky pro ladění a opravu	10
7	Uživatelská příručka	11
7.1	Hlavní menu	11
7.2	Nastavení	11
7.3	Hra (PlayingState)	12
8	Závěr	12
9	Zdroje	13
9.1	Další potenciální zdroje inspirace a informací	14

1 Cíl práce

Cílem projektu bylo vytvořit 2D počítačovou hru inspirovanou klasikou "Lunar Lander". Hráč ovládá lunární modul s omezeným množstvím paliva a jeho úkolem je bezpečně přistát na povrchu Měsíce, popřípadě na přistávacích plošinách s násobky bodů. Projekt by měl obsahovat následující klíčové funkce:

- Realistickou (v rámci 2D arkády) fyziku pohybu modulu, zahrnující gravitaci, tah hlavního motoru a rotační trysky.
- Správu paliva, které se spotřebovává používáním motoru.
- Detekci kolizí modulu s terénem a přistávacími plošinami.
- Systém hodnocení kvality přistání (havárie, tvrdé přistání, perfektní přistání) ovlivňující skóre.
- Několik herních stavů: hlavní menu, samotná hra a obrazovka nastavení.
- Možnost upravit hlasitost zvuků a změnit ovládací klávesy.
- Zvukové efekty pro akce ve hře.
- Procedurálně generovaný terén s různě bodovanými přistávacími plošinami.

2 Popis práce

Program implementuje hru Lunar Lander, kde hráč manévruje s kosmickou lodí, snaží se šetřit palivo a bezpečně přistát na Měsíci. Hra je napsána v jazyce Java a využívá standardní knihovny pro grafiku a zvuk.

2.1 Herní mechaniky

- **Ovládání modulu:** Hráč ovládá tah hlavního motoru (proti gravitaci) a dvě rotační trysky pro natáčení modulu doleva a doprava.
- **Fyzika:** Pohyb modulu je ovlivněn konstantní lunární gravitací (`LUNAR_GRAVITY_MS2`). Hlavní motor poskytuje zrychlení proti směru orientace modulu (`MAIN_THRUSTER_ACCELERATION_MS2`). Rotace probíhá konstantní rychlostí (`ROTATION_DEGREES_PER_SECOND`).
- **Palivo:** Modul startuje s maximálním množstvím paliva (`MAX_FUEL`). Používání hlavního motoru spotřebovává palivo (`FUEL_CONSUMPTION_RATE_PER_SECOND_AT_FULL_THRUST`). Pokud palivo dojde, motor přestane fungovat.
- **Terén a přistávací plošiny:** Terén je generován na základě předdefinovaného vzoru (`RAW_TERRAIN_DATA` v `Terrain.java`) a opakuje se, přičemž se na něm náhodně objevují přistávací plošiny s různými násobiteli skóre.
- **Přistání a havárie:** Kontakt s terénem je vyhodnocen na základě kolize sady bodů na modulu (podvozek, špička) s polygonem terénu. Kvalita přistání závisí na vertikální a horizontální rychlosti a úhlu modulu při dopadu. Hranice pro havárii jsou

CRASH_LIMIT_SPEED_Y, CRASH_LIMIT_SPEED_X a CRASH_LIMIT_ANGLE_DEG. Perfektní přistání vyžaduje rychlosti pod PERFECT_LANDING_SPEED_Y, PERFECT_LANDING_SPEED_X a úhel pod PERFECT_LANDING_ANGLE_DEG. Havárie vede k nulovému skóre a penalizaci paliva.

- **Skóre:** Skóre se počítá na základě kvality přistání a násobitele přistávací plošiny. Celkové skóre se kumuluje napříč pokusy, dokud nedojde palivo nebo hráč neukončí hru.
- **Zvuk:** Hra obsahuje zvukové efekty pro kliknutí tlačítek, chod motoru, explozi a smrt hráče (vyčerpání paliva po úspěšném přistání). Hlasitost lze globálně regulovat.
- **Kamera:** Kamera sleduje modul, dynamicky scrolluje a přibližuje/oddaluje podle výšky modulu nad terénem (DISTANCE_TO_ZOOM_IN_THRESHOLD, DISTANCE_TO_ZOOM_OUT_THRESHOLD).

2.2 Uživatelské rozhraní a průběh hry

Hra nabízí grafické uživatelské rozhraní.

- **Hlavní menu:** Zobrazuje název hry "Lunar Lander" a volby:
 - Spustit hru (Enter)
 - Nastavení (S)
 - Ukončit (Esc)
- **Herní obrazovka (PlayingState):** Zobrazuje letící modul nad terénem. V horní části je HUD (Heads-Up Display) s informacemi:
 - SKÓRE: Celkové dosažené skóre.
 - ČAS: Uplynulý čas aktuálního pokusu.
 - PALIVO: Zbývajících množství paliva.
 - VÝŠKA: Aktuální výška modulu nad terénem.
 - HOR.RYCHL: Horizontální rychlost modulu.
 - VER.RYCHL: Vertikální rychlost modulu.

Po přistání nebo havárii se zobrazí zpráva (např. "Perfektní přistání!", "Havárie!") a případné získané body. Hráč může pokračovat (Enter) nebo se vrátit do menu (Esc).

- **Nastavení (SettingsState):** Umožňuje:
 - Nastavení hlasitosti pomocí posuvníku.
 - Změnu ovládacích kláves pro akce: Rotace vlevo, Rotace vpravo, Tah motoru.Návrat do menu se provádí klávesou Esc.
- **Výstup v konzoli:** Během běhu hry se v konzoli mohou objevit výpisy aktuálních hodnot FPS a UPS (např. FPS: 60, UPS: 60). Dále se zde mohou zobrazit chybová hlášení, například při problémech s načítáním zvukových či obrazových souborů (např. Chyba: Zvukový soubor nenalezen - /sounds/engine.wav).

2.3 Algoritmy

- **Pohyb modulu:** Výpočet pozice a rychlosti modulu využívá jednoduchou Eulerovu integraci, kde se v každém časovém kroku (`deltaTime`) aktualizuje rychlost na základě zrychlení (gravitace, tah motoru) a následně pozice na základě nové rychlosti.
- **Generování terénu:** Terén je generován z řetězce `RAW_TERRAIN_DATA` obsahujícího souřadnice bodů. Tyto body jsou transformovány (normalizace, škálování na osu X a Y dle `X_AXIS_SHRINK_FACTOR` a mapování na viewport dle `TERRAIN_VIEWPORT_TOP_Y_RATIO` a `TERRAIN_VIEWPORT_BOTTOM_Y_RATIO`) a vytvářejí základní vzor, který se opakuje. Přistávací plošiny jsou definovány v rámci tohoto vzoru a náhodně se zobrazují s pravděpodobností `PAD_SPAWN_PROBABILITY`.
- **Detekce kolizí:** Modul má definované kolizní body (dva na podvozku, jeden na špičce). V každém kroku se kontroluje, zda některý z těchto bodů (transformovaných do světových souřadnic) leží uvnitř polygonu terénu (`terrainPolygon.contains(worldP)`).
- **Výpočet skóre:** Skóre je odvozeno z kvality přistání, která je funkcí rychlostí (V_x , V_y) a úhlu při dopadu. Tyto hodnoty se porovnávají s prahy pro perfektní přistání a havárii. Výsledná kvalita (0-1) se násobí základním skóre (`MAX_BASE_SCORE_FOR_QUALITY_1`) a násobitelem plošiny.

3 Systémové požadavky (System requirements)

- **Programovací jazyk:** Java.
- **Požadovaná verze JDK:** JDK 23 (dle konfiguračního souboru projektu `misc.xml`, konkrétně `languageLevel="JDK_23"` a `project-jdk-name="openjdk-23"`).
- **Knihovny:**
 - Standardní Java knihovny:
 - * AWT a Swing pro grafické rozhraní a zpracování vstupů (`java.awt`, `javax.swing`).
 - * `javax.sound.sampled` pro přehrávání zvuků.
 - Nejsou použity žádné externí knihovny třetích stran.
- **Operační systém:** Libovolný OS s nainstalovaným kompatibilním JDK (Windows, macOS, Linux).

4 Základní struktura

Program je navržen objektově a rozdělen do několika balíčků a tříd s jasně definovanými zodpovědnostmi.

- **Hlavní třídy a balíčky:**

- `Main.java`: Vstupní bod aplikace, vytváří instanci třídy `Game` a spouští hru.
- Balíček `core`:
 - * `Game.java`: Obsahuje hlavní herní smyčku (`run()`), spravuje časování (FPS/UPS), inicializuje a řídí ostatní hlavní komponenty jako `Window`, `InputHandler` a `StateManager`.
 - * `Window.java`: Zajišťuje vytvoření a správu okna hry (`JFrame`) a kreslí plochy (`Canvas`), včetně buffer strategy pro plynulé vykreslování.
- Balíček `input`:
 - * `InputHandler.java`: Zpracovává vstupy z klávesnice a myši (`KeyListener`, `MouseListener`, `MouseMotionListener`). Spravuje aktuální a předchozí stavy kláves/myši.
 - * `KeyBindings.java`: Mapuje herní akce (`GameAction`) na konkrétní kódy kláves (`KeyEvent`). Umožňuje změnu přiřazení kláves.
 - * `GameAction.java`: Výčtový typ (`enum`) definující možné herní akce (např. `ROTATE_LEFT`, `THRUST`).
- Balíček `states`:
 - * `StateManager.java`: Správce herních stavů. Řídí přechody mezi různými stavy hry (`MenuState`, `PlayingState`, `SettingsState`).
 - * `GameState.java`: Rozhraní definující základní metody pro všechny herní stavy (`init`, `onEnter`, `update`, `render`, `handleInput`, `onExit`).
 - * `MenuState.java`, `PlayingState.java`, `SettingsState.java`: Konkrétní implementace herních stavů.
- Balíček `entities`:
 - * `Lander.java`: Reprezentuje lunární modul. Obsahuje logiku pro fyziku pohybu, spotřebu paliva, detekci kolizí, stavy modulu (let, přistání, havárie) a jeho vykreslení.
 - * `Terrain.java`: Generuje a vykresluje herní terén, včetně přistávacích plošin. Poskytuje kolizní polygon terénu.
 - * `LandingPad.java`: Reprezentuje přistávací plošinu s násobitelem skóre.
- Balíček `audio`:
 - * `AudioManager.java`: Singleton třída pro načítání a přehrávání zvukových efektů. Umožňuje nastavení globální hlasitosti.
- Balíček `ui`:
 - * `Slider.java`: UI komponenta pro posuvník, použitá v nastavení pro regulaci hlasitosti.
- **Komunikace mezi komponentami:**
 - `Game` inicializuje a volá metody `StateManager`, `InputHandler` a `Window`.
 - `StateManager` přepíná aktivní `GameState` a volá jeho metody.
 - Jednotlivé stavy (`MenuState`, `PlayingState`, `SettingsState`) využívají

`InputHandler` pro reakci na uživatelské vstupy a `AudioManager` pro přehrávání zvuků.

- `PlayingState` spravuje instance `Lander` a `Terrain`. `Lander` interaguje s `Terrain` pro detekci kolizí.
- `SettingsState.java` komunikuje s `AudioManager` (změna hlasitosti) a s `KeyBindings` (přes `InputHandler`) pro změnu ovládání.

5 Testovací data

Pro zajištění správné funkčnosti programu je doporučeno kombinovat jednotkové testy (JUnit) s manuálním testováním různých herních scénářů.

5.1 Jednotkové testy (Unit Tests)

Projekt obsahuje sadu JUnit testů pro klíčové komponenty:

- `AudioManagerTest.java`: Testuje správnou inicializaci singletonu, omezování hlasitosti na platné rozsahy (0–100 %) a funkčnost zvyšování/snižování hlasitosti.
- `KeyBindingsTest.java`: Ověřuje načtení výchozích klávesových zkratk, nastavení nových kláves, řešení konfliktů při přiřazení již použité klávesy a správné generování textového popisu klávesy.
- `LanderTest.java`: Testuje počáteční stav modulu (pozice, rychlost, palivo), spotřebu paliva při použití motoru a logiku rotace. Obsahuje také poznámky o koncepčních testech pro havárie a výpočet skóre, které by vyžadovaly pokročilejší techniky (mocking) pro izolované testování.
- `SliderTest.java`: Prověřuje počáteční hodnotu posuvníku, omezování hodnot na definovaný rozsah, správné nastavení hodnoty a reakci na simulovaný vstup myši (drag and drop).
- `StateManagerTest.java`: Testuje počáteční stav (žádný aktivní stav), správné přepínání mezi definovanými stavy (`Menu`, `Playing`, `Settings`) a kontroluje, zda jsou stavy správných typů.

5.2 Manuální testování

Manuální testování by se mělo zaměřit na komplexní herní scénáře a interakce:

- **Herní scénáře:**
 - Úspěšné perfektní přistání na různých typech plošin (s různými násobiteli skóre).
 - Úspěšné tvrdé přistání.
 - Havárie způsobená překročením povolené vertikální rychlosti.
 - Havárie způsobená překročením povolené horizontální rychlosti.
 - Havárie způsobená příliš velkým náklonem modulu.

- Havárie nárazem do svahu terénu nebo pádem mimo obrazovku.
- Vyčerpání paliva během letu a následný pád.
- Úspěšné přistání s minimálním nebo žádným zbývajícím palivem (ověření zprávy o konci hry).
- **Ovládání a UI:**
 - Funkčnost výchozích ovládacích kláves (W, A, D).
 - Procházení všemi volbami v hlavním menu (Spustit hru, Nastavení, Ukončit).
 - V obrazovce Nastavení:
 - * Regulace hlasitosti pomocí posuvníku a ověření změny úrovně zvuku.
 - * Změna přiřazení kláves pro všechny herní akce; testovat i přiřazení již obsazené klávesy (řešení konfliktu) a speciálních kláves.
 - * Zrušení změny klávesy pomocí ESC.
 - * Ověření funkčnosti nově nastavených kláves ve hře.
- **HUD a zpětná vazba:**
 - Správnost zobrazovaných údajů na HUD (skóre, čas, palivo, rychlosti, výška).
 - Zobrazení korektních zpráv po skončení pokusu (přistání/havárie) a správný výpočet skóre.
 - Funkčnost zvukových efektů (motor, kliknutí, exploze).
 - Chování kamery (scrollování, přibližování/oddalování).

Důvodem pro testování těchto funkcí je ověření robustnosti herní logiky, správnosti fyzikálního modelu, intuitivnosti ovládání a celkového herního zážitku.

6 Analýza problému nefunkčního unit testu

Během vývoje a testování projektu může dojít k situaci, kdy některý z jednotkových testů selže. Následující text demonstruje analýzu konkrétního selhání unit testu pro komponentu `Slider.java`, jak bylo indikováno výstupem z testovacího frameworku.

6.1 Popis chyby z výstupu testu

Testovací framework JUnit ohlásil chybu typu `org.opentest4j.AssertionFailedError` v testovací třídě `SliderTest.java`, konkrétně v metodě `testUpdateValueFromMouseCalculatesCorrectly` na řádce 72. Chybová zpráva byla:

```
org.opentest4j.AssertionFailedError: Slider value should be 0 when mouse is at the beginning
Expected :0
Actual   :2
<Click to see difference>
```



```
at org.junit.jupiter.api.AssertionsUtils.fail(AssertionsUtils.java:55)
at org.junit.jupiter.api.AssertionsUtils.failNotEqual(AssertionsUtils.java:62)
at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:150)
at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:559)
at SliderTest.testUpdateValueFromMouseCalculatesCorrectly(SliderTest.java:72)
at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:117)
at java.base/java.lang.reflect.Method.invoke(Method.java:580)
at org.junit.platform.commons.util.ReflectionUtils.invokeMethod(ReflectionUtils.java:725)
at org.junit.jupiter.engine.execution.MethodInvocation.proceed(MethodInvocation.java:60)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain$ValidatingInvocation.proceed(InvocationInterceptorChain.java:131)
at org.junit.jupiter.engine.extension.TimeoutExtension.intercept(TimeoutExtension.java:149)
at org.junit.jupiter.engine.extension.TimeoutExtension.interceptTestableMethod(TimeoutExtension.java:160)
at org.junit.jupiter.engine.extension.TimeoutExtension.interceptTestMethod(TimeoutExtension.java:843)
at org.junit.jupiter.engine.execution.ExecutableInvoker$ReflectiveInterceptorCall.lambda$ofVoidMethod$0(ExecutableInvoker.java:133)
at org.junit.jupiter.engine.execution.ExecutableInvoker.lambda$invoke$0(ExecutableInvoker.java:106)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain$InterceptedInvocation.proceed(InvocationInterceptorChain.java:106)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain.proceed(InvocationInterceptorChain.java:64)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain.chainAndInvoke(InvocationInterceptorChain.java:45)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain.invoke(InvocationInterceptorChain.java:39)
at org.junit.jupiter.engine.execution.ExecutableInvoker.invoke(ExecutableInvoker.java:117)
at org.junit.jupiter.engine.execution.ExecutableInvoker.invoke(ExecutableInvoker.java:106)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor.lambda$invokeTestMethod$7(TestMethodTestDescriptor.java:218)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(ThrowableCollector.java:73)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor.invokeTestMethod(TestMethodTestDescriptor.java:214)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor.execute(TestMethodTestDescriptor.java:135)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor.execute(TestMethodTestDescriptor.java:62)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursively$6(NodeTestTask.java:151)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(ThrowableCollector.java:73)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursively$8(NodeTestTask.java:141)
at org.junit.platform.engine.support.hierarchical.Node.around(Node.java:137)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursively$9(NodeTestTask.java:146)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(ThrowableCollector.java:73)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.executeRecursively(NodeTestTask.java:135)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.execute(NodeTestTask.java:95)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1597)
at org.junit.platform.engine.support.hierarchical.SameThreadHierarchicalTestExecutor.run(SameThreadHierarchicalTestExecutor.java:40)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursively$6(NodeTestTask.java:151)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(ThrowableCollector.java:73)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursively$8(NodeTestTask.java:141)
at org.junit.platform.engine.support.hierarchical.Node.around(Node.java:137)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursively$9(NodeTestTask.java:146)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(ThrowableCollector.java:73)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.executeRecursively(NodeTestTask.java:135)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.execute(NodeTestTask.java:95)
```

```

at org.junit.platform.engine.support.hierarchical.Node.around(Node.java:137)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursiv
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(Throwable
at org.junit.platform.engine.support.hierarchical.NodeTestTask.executeRecursively(Noc
at org.junit.platform.engine.support.hierarchical.NodeTestTask.execute(NodeTestTask.
at java.base/java.util.ArrayList.forEach(ArrayList.java:1597)
at org.junit.platform.engine.support.hierarchical.SameThreadHierarchicalTestExecuto
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursiv
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(Throwable
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursiv
at org.junit.platform.engine.support.hierarchical.Node.around(Node.java:137)
at org.junit.platform.engine.support.hierarchical.NodeTestTask.lambda$executeRecursiv
at org.junit.platform.engine.support.hierarchical.ThrowableCollector.execute(Throwable
at org.junit.platform.engine.support.hierarchical.NodeTestTask.executeRecursively(Noc
at org.junit.platform.engine.support.hierarchical.NodeTestTask.execute(NodeTestTask.
at org.junit.platform.engine.support.hierarchical.SameThreadHierarchicalTestExecuto
at org.junit.platform.engine.support.hierarchical.HierarchicalTestExecutor.execute(H
at org.junit.platform.engine.support.hierarchical.HierarchicalTestEngine.execute(Hier
at org.junit.platform.launcher.core.EngineExecutionOrchestrator.execute(EngineExecut
at org.junit.platform.launcher.core.EngineExecutionOrchestrator.execute(EngineExecut
at org.junit.platform.launcher.core.EngineExecutionOrchestrator.lambda$execute$0(Eng
at org.junit.platform.launcher.core.EngineExecutionOrchestrator.withInterceptedStrea
at org.junit.platform.launcher.core.EngineExecutionOrchestrator.execute(EngineExecut
at org.junit.platform.launcher.core.DefaultLauncher.execute(DefaultLauncher.java:114)
at org.junit.platform.launcher.core.DefaultLauncher.execute(DefaultLauncher.java:86)
at org.junit.platform.launcher.core.DefaultLauncherSession$DelegatingLauncher.execute
at org.junit.platform.launcher.core.SessionPerRequestLauncher.execute(SessionPerRequ
at com.intellij.junit5.JUnit5IdeaTestRunner.startRunnerWithArgs(JUnit5IdeaTestRunner
at com.intellij.rt.junit.IdeaTestRunner$Repeater$1.execute(IdeaTestRunner.java:38)
at com.intellij.rt.execution.junit.TestsRepeater.repeat(TestsRepeater.java:11)
at com.intellij.rt.junit.IdeaTestRunner$Repeater.startRunnerWithArgs(IdeaTestRunner.
at com.intellij.rt.junit.JUnitStarter.prepareStreamsAndStart(JUnitStarter.java:231)
at com.intellij.rt.junit.JUnitStarter.main(JUnitStarter.java:55)

```

Tento výstup jasně ukazuje, že test očekával hodnotu posuvníku 0, ale aktuálně zjištěná hodnota byla 2. Test tedy kontroloval, zda při umístění myši na samotný začátek aktivní plochy posuvníku bude jeho vypočtená hodnota minimální (v tomto případě 0).

6.2 Pravděpodobné příčiny problému

Selhání testu `testUpdateValueFromMouseCalculatesCorrectly` naznačuje, že metoda v třídě `Slider`, zodpovědná za převod pozice myši na hodnotu posuvníku, nepracuje správně pro krajní případ minimální hodnoty. Možné příčiny zahrnují:

- **Off-by-one/Off-by-two error:** Běžná chyba při výpočtech zahrnujících rozsahy nebo indexy, kde výsledek může být posunut o jednu či více jednotek. V tomto případě je aktuální hodnota o 2 vyšší, než se očekávalo.
- **Chybné zpracování okrajových podmínek:** Logika výpočtu nemusí správně ošetřovat situaci, kdy je kurzor myši přesně na začátku (nebo konci) dráhy posuvníku.
- **Vliv odsazení (padding/margin) nebo šířky samotného jezdce:** Pokud má posuvník nebo jeho jezdec nějaké vizuální odsazení, nebo pokud se při výpočtu pozice nezohledňuje šířka samotného jezdce (např. počítá se s levým okrajem jezdce místo jeho středu), může to vést k nepřesnému mapování pozice myši na hodnotu. Například, pokud je minimální pozice myši pro hodnotu 0 interpretována jako pozice o několik pixelů dále.
- **Zaokrouhlovací chyby nebo celočíselné dělení:** Pokud výpočet zahrnuje převod z plovoucí desetinné čárky na celé číslo nebo používá celočíselné dělení, může docházet k neočekávanému zaokrouhlení nebo oříznutí desetinné části, které způsobí posun výsledné hodnoty.
- **Nesprávná interpretace rozměrů komponenty:** Výpočet může vycházet z nesprávné šířky aktivní plochy posuvníku. Například, pokud je šířka získána z celé komponenty, ale aktivní dráha pro myš je menší kvůli okrajům.
- **Problém s referenčním bodem:** Výpočet může nesprávně určovat nulový bod (počátek) dráhy posuvníku vzhledem k pozici myši.

6.3 Možné kroky pro ladění a opravu

Pro identifikaci přesné příčiny a opravu chyby by bylo vhodné provést následující kroky:

1. **Detailní revize kódu:** Pečlivě prozkoumat metodu v `Slider.java`, která je zodpovědná za výpočet hodnoty na základě pozice myši (pravděpodobně metoda volaná z `update` nebo obsluhující události myši). Zaměřit se na matematické operace, použití rozměrů komponenty a zpracování hranic.
2. **Logování hodnot:** Dočasně přidat do metody výpisy klíčových proměnných:
 - Aktuální souřadnice myši.
 - Rozměry posuvníku (šířka, pozice x).
 - Vypočtená relativní pozice myši vůči posuvníku.
 - Mezivýsledky výpočtu hodnoty před zaokrouhlením nebo typovou konverzí.
 - Finální vypočtená hodnota.

Toto logování by mělo být aktivní při spuštění konkrétního selhávajícího testu.

3. **Krokování pomocí debuggeru:** Využít debugger v IDE k procházení kódu řádek po řádku během provádění testu `testUpdateValueFromMouseCalculatesCorrectly`. To umožní sledovat stav proměnných v reálném čase a identifikovat místo, kde výpočet odbočuje od očekávaného chování.
4. **Izolace výpočetní logiky:** Pokud je výpočet komplexní, je možné jej dočasně izolovat do samostatné testovací metody nebo třídy a testovat s přesně definovanými vstupy, které odpovídají selhávajícímu scénáři.
5. **Ověření předpokladů testu:** Zkontrolovat, zda test správně nastavuje podmínky pro testovaný scénář – například, zda simulovaná pozice myši ("at the beginning") skutečně odpovídá souřadnici, která by měla produkovat hodnotu 0 (např. x-ová souřadnice posuvníku). Může zde být nesoulad mezi tím, co test považuje za "začátek", a jak to interpretuje logika komponenty.
6. **Zvážit geometrii posuvníku:** Přemýšlet o tom, jak je definován "začátek" posuvníku. Je to levý okraj celé komponenty, nebo levý okraj dráhy, po které se pohybuje jezdec? Je šířka jezdce zohledněna? Například, pokud je hodnota mapována na střed jezdce, pak když je levý okraj jezdce na pixelu 0, jeho střed (a tedy i hodnota) může být mírně posunut.

Systematickým uplatněním těchto kroků by mělo být možné odhalit zdroj chyby a upravit logiku výpočtu hodnoty v komponentě `Slider` tak, aby test procházel úspěšně a komponenta se chovala korektně i v okrajových případech.

7 Uživatelská příručka

Tato příručka popisuje, jak ovládat hru Lunar Lander.

7.1 Hlavní menu

Po spuštění hry se zobrazí hlavní menu s následujícími možnostmi:

- **Spustit hru:** Stiskněte klávesu `ENTER`.
- **Nastavení:** Stiskněte klávesu `S`.
- **Ukončit:** Stiskněte klávesu `ESC`.

7.2 Nastavení

V obrazovce nastavení můžete upravit:

- **Hlasitost:** Pomocí myši klikněte a táhněte posuvník "Hlasitost" pro úpravu celkové hlasitosti hry.
- **Ovládání:** Klikněte myší na herní akci, jejíž klávesu chcete změnit (např. "Rotace vlevo"). Poté stiskněte novou požadovanou klávesu. Pokud chcete změnu zrušit, stiskněte `ESC` během čekání na novou klávesu. Výchozí ovládání je:

- W - Tah motoru
- A - Rotace vlevo
- D - Rotace vpravo
- Pro návrat do hlavního menu stiskněte klávesu **ESC**.

7.3 Hra (PlayingState)

- **Ovládání modulu:** Použijte nastavené klávesy pro tah motoru, rotaci vlevo a rotaci vpravo.
- **Cíl:** Bezpečně přistát na rovných přistávacích plošinách. Jemné přistání (nízká rychlost, malý náklon) přináší více bodů. Plošiny mohou mít různé bodové násobitele.
- **HUD (Heads-Up Display):** Sledujte informace v horní části obrazovky:
 - **SKÓRE:** Vaše celkové skóre v aktuální herní seanci.
 - **ČAS:** Čas uplynulý v tomto pokusu o přistání.
 - **PALIVO:** Zbývajícím palivo. Tah motoru spotřebovává palivo. Pokud palivo dojde, nelze použít motor.
 - **VÝŠKA:** Výška spodní části modulu nad terénem přímo pod ním.
 - **HOR.RYCHL:** Horizontální rychlost modulu.
 - **VER.RYCHL:** Vertikální rychlost modulu.
- **Přistání/Havárie:**
 - Příliš vysoká rychlost nebo velký náklon při kontaktu s terénem způsobí "Havárii!" nebo "Tvrdé přistání!".
 - Úspěšné a jemné přistání je označeno jako "Perfektní přistání!".
- **Po skončení pokusu:**
 - Pokud máte stále palivo, stiskněte **ENTER** pro další pokus (zbývajícím palivo a skóre se přenáší).
 - Stiskněte **ESC** pro návrat do hlavního menu (aktuální herní seance končí).
 - Pokud palivo došlo, hra končí a je nutné se vrátit do menu klávesou **ESC**.
- Kdykoliv během hry můžete stisknout **ESC** pro okamžitý návrat do hlavního menu (aktuální pokus a seance budou ukončeny).

8 Závěr

Vývoj hry Lunar Lander byl zajímavou zkušeností, která přinesla několik výzev i poučení.

Problémy a výzvy během tvorby:

- **Ladění fyziky:** Nalezení správné rovnováhy mezi gravitací, silou motoru, rychlostí rotace a spotřebou paliva tak, aby byl modul ovladatelný, ale hra stále představovala výzvu, vyžadovalo iterativní testování a úpravy konstant.
- **Detekce kolizí:** Implementace přesné a zároveň výpočetně nenáročné detekce kolizí

mezi rotujícím modulem (reprezentovaným několika body) a dynamicky generovaným polygonem terénu.

- **Návrh skórovacího systému:** Vytvoření systému, který spravedlivě odměňuje kvalitu přistání a zohledňuje různé faktory (rychlosti, úhel, násobitel plošiny), bylo klíčové pro motivaci hráče.
- **Kamera a zoom:** Implementace plynulého pohybu kamery, která sleduje modul a zároveň se inteligentně přibližuje/oddaluje v závislosti na výšce nad terénem, byla důležitá pro přehlednost.
- **Generování terénu:** Transformace dat terénu a zajištění plynulého napojování opakujícího se vzoru při pohybu kamery.
- **Jednotkové testování:** Testování částí herní logiky úzce spjatých s vykreslováním nebo komplexními interakcemi objektů (např. `LanderTest` pro kolize) bylo obtížné bez pokročilejších technik (mocking), jak je zmíněno v komentářích testů.

Zavrhnuté implementace (hypoteticky):

- Mohla být zvažována jednodušší reprezentace modulu jako jednoho bodu, ale pro lepší vizuální dojem a přesnější kolize byl zvolen model s definovaným tvarem a více kolizními body.
- Původní systém terénu mohl být čistě náhodný, ale pro lepší kontrolu nad hratelností a možností umístění specifických plošin byl zvolen přístup s opakujícím se vzorem definovaným daty.

Celkový průběh a dojem z práce: Projekt úspěšně implementuje klíčové mechaniky hry Lunar Lander. Nabízí kompletní herní smyčku od hlavního menu, přes samotnou hru, nastavení, až po vyhodnocení pokusů a návrat do menu. Funkce jako kumulativní skóre, přenos paliva mezi pokusy (pokud nedošlo), detailní HUD a možnost konfigurace přispívají k celkovému hernímu zážitku. Kód je strukturován do logických celků (balíčků a tříd), což usnadňuje orientaci a případné další rozšíření. Práce na projektu byla přínosná pro prohloubení znalostí v oblasti objektově orientovaného programování v Javě, základů herní fyziky, správy stavů a návrhu uživatelského rozhraní.

9 Zdroje

Při vývoji tohoto projektu byly primárně využity standardní knihovny a funkce jazyka Java.

1. Oracle. *Java® Platform, Standard Edition & Java Development Kit Version 23 API Specification*. Dostupné online: <https://docs.oracle.com/en/java/javase/23/docs/api/index.html> (Přístup závisí na aktuální dokumentaci JDK 23).
2. Pro obecné principy hry "Lunar Lander" bylo čerpáno z veřejně dostupných informací o této klasické hře.
3. Pro rozsáhlejší konzultace a řešení komplexnějších problémů a otázek bylo použito

Gemini AI.

4. Podkladové studijní materiály pro výuku prvního a druhého ročníku IT a pro výuku druhého ročníku TI.

Žádné externí knihovny třetích stran (mimo standardní JDK) nebyly použity. Konkrétní úryvky kódu nebo specifické algoritmy z externích zdrojů nebyly přebírány bez uvedení, pokud by takový případ nastal.

9.1 Další potenciální zdroje inspirace a informací

Kromě přímo uvedených zdrojů mohly být při obecném vývoji podobného projektu nebo pro hlubší studium problematiky relevantní i následující typy zdrojů:

- **Online komunity a fóra:** Webové stránky jako Stack Overflow, Reddit (např. r/gamedev, r/java), nebo specifická fóra zaměřená na vývoj her v Javě mohly poskytnout řešení konkrétních programátorských problémů nebo nabídnout různé perspektivy.
- **Tutoriály a blogy o vývoji her:** Mnoho webových stránek a blogů (např. dříve Gamasutra, nyní Game Developer, různé osobní blogy vývojářů) nabízí tutoriály, články o designu her, fyzice ve hrách, nebo implementaci specifických herních mechanik.
- **Knihy o programování v Javě a vývoji her:** Existuje řada publikací zabývajících se pokročilými tématy v Javě, objektově orientovaným návrhem, stejně jako knihy přímo o principech vývoje her, herní fyzice, či umělé inteligenci ve hrách.
- **Dokumentace k Java AWT/Swing:** Pro detailní pochopení možností a chování grafických komponent mohla být užitečná podrobnější dokumentace a příklady nad rámec základní API specifikace.
- **Open-source projekty:** Studium existujících open-source implementací Lunar Landeru nebo podobných her v Javě mohlo poskytnout inspiraci pro řešení určitých mechanik či struktury kódu (nikoliv však pro přímé kopírování).
- **Univerzitní kurzy a materiály:** Materiály z univerzitních kurzů zaměřených na programování, softwarové inženýrství nebo vývoj her často obsahují relevantní teoretické i praktické informace.

Tyto zdroje by sloužily k rozšíření obecných znalostí a k nalezení řešení pro specifické výzvy, které se mohou během vývoje vyskytnout.